

EECE527
INTRODUCTION TO HAPTICS
LECTURE 13: COLLISION DETECTION (4)



2011-04-21
Seungmoon Choi, Ph.D.

BOUNDING VOLUMES

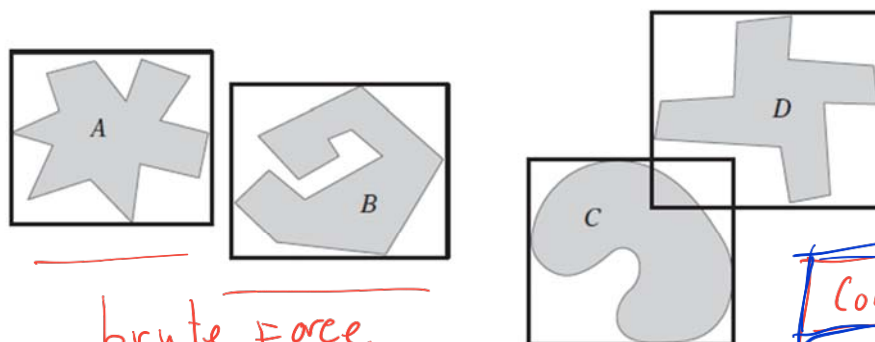
Bounding Volume

- ▶ A **bounding volume** (BV) is a single simple volume encapsulating one or more objects of more complex nature.
- ▶ Using bounding volumes allows for fast overlap rejection tests.

Unstoth

CUDA

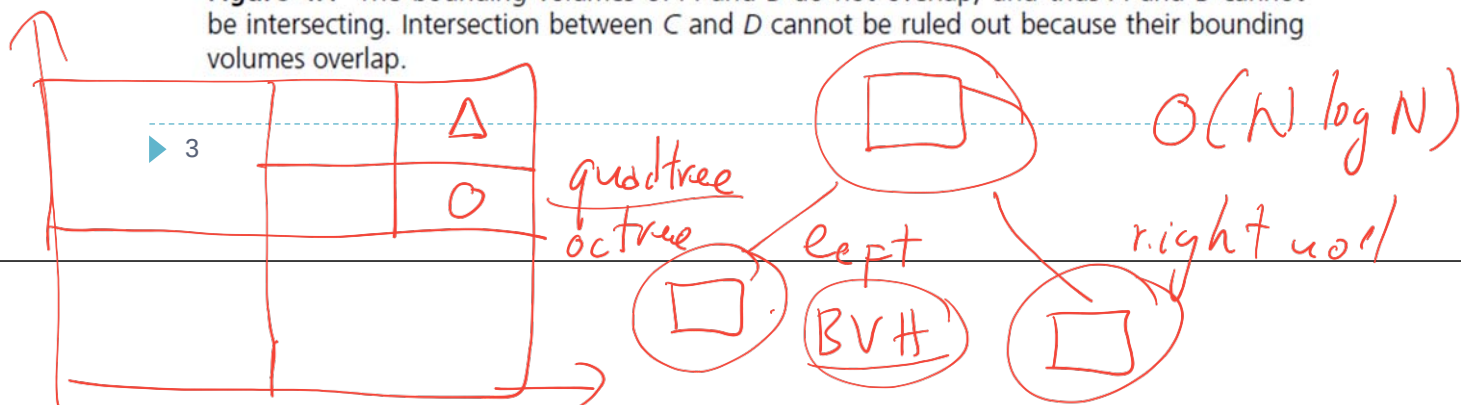
pytorch
tensorflow



$O(N^2)$

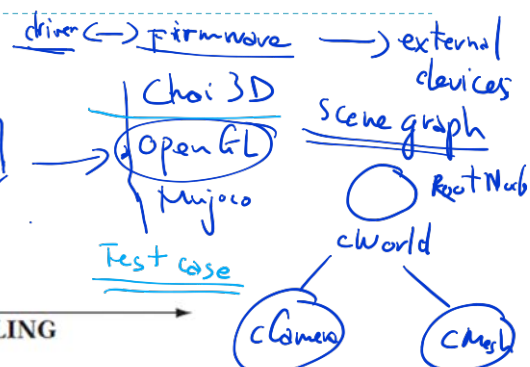
Collision Detection!

Figure 4.1 The bounding volumes of A and B do not overlap, and thus A and B cannot be intersecting. Intersection between C and D cannot be ruled out because their bounding volumes overlap.



Desirable BV Characteristics

- ▶ Inexpensive intersection tests
- ▶ Tight fitting
- ▶ Inexpensive to compute
- ▶ Easy to rotate and transform
- ▶ Use little memory



BETTER BOUND, BETTER CULLING

FASTER TEST, LESS MEMORY

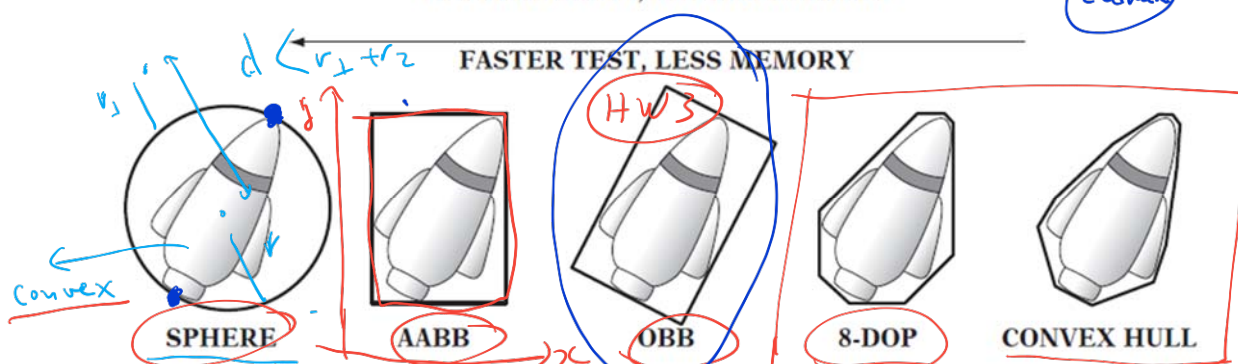


Figure 4.2 Types of bounding volumes: sphere, axis-aligned bounding box (AABB), oriented bounding box (OBB), eight-direction discrete orientation polytope (8-DOP), and convex hull.

Axis-aligned Bounding Boxes (AABBs)

- ▶ A rectangular six-sided box (in 3D, four-sided in 2D) categorized by having its faces oriented in such a way that its face normals are always parallel with the axes of the given coordinate system.
- ▶ Lead to very fast overlap checks
- ▶ One of the most common bounding volumes

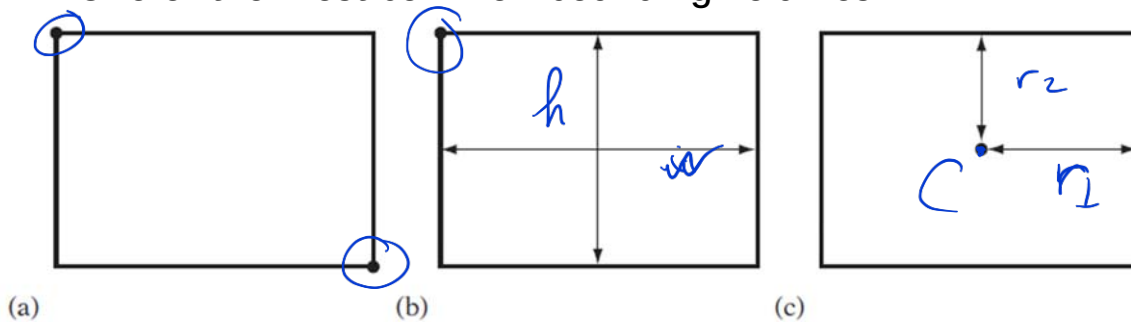


Figure 4.3 The three common AABB representations: (a) min-max, (b) min-widths, and (c) center-radius.

▶ 5

class AABB
{ float[3] min
 float[3] max

class AABB
{ float[3] min
 float[3] hwe

AABB-AABB Intersection

- ▶ Very straightforward
- ▶ Two AABBs overlap only if they overlap on all three axes, where their extent along each dimension is seen as an interval on the corresponding axis.

min - max

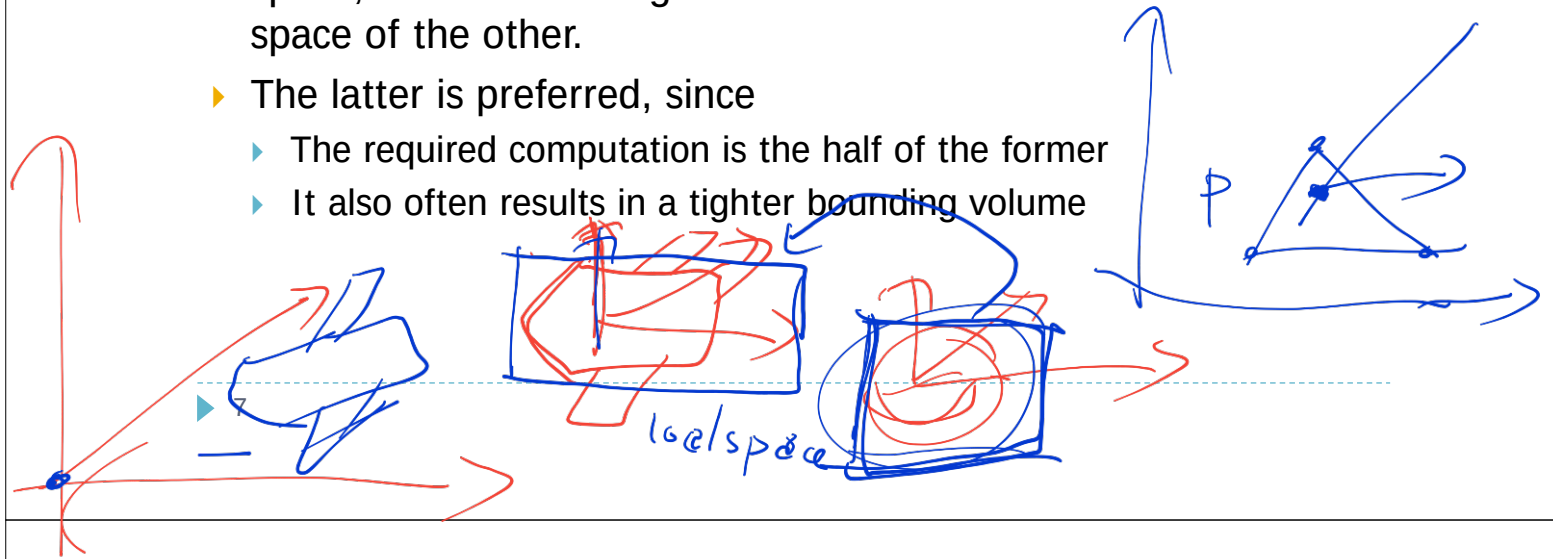
```
int TestAABBAABB(AABB a, AABB b)
{
    // Exit with no intersection if separated along an axis
    if (a.max[0] < b.min[0] || a.min[0] > b.max[0]) return 0;
    if (a.max[1] < b.min[1] || a.min[1] > b.max[1]) return 0;
    if (a.max[2] < b.min[2] || a.min[2] > b.max[2]) return 0;
    // Overlapping on all axes means AABBs are intersecting
    return 1;
}
```

Choi 3D
Munjoco

▶ 6

Computing and Updating AABBs

- ▶ Bounding volumes are usually specified in the local model space of the objects they bound.
- ▶ To perform an overlap query between two bounding volumes, the volumes must be transformed into a common coordinate system.
- ▶ The two bounding volumes are transformed into the world space, or one bounding volume is transformed into the local space of the other.
- ▶ The latter is preferred, since
 - ▶ The required computation is the half of the former
 - ▶ It also often results in a tighter bounding volume



Computing and Updating AABBs - Concept

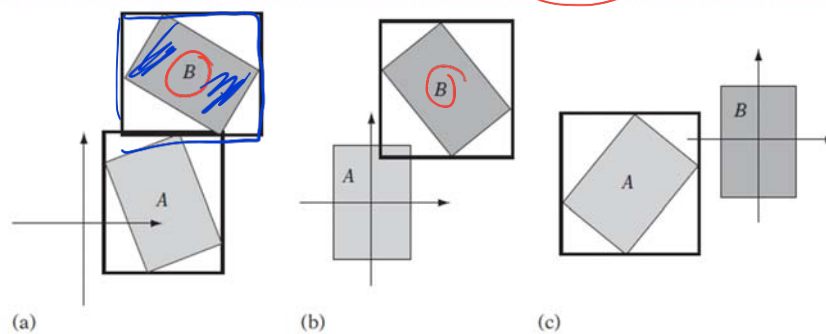
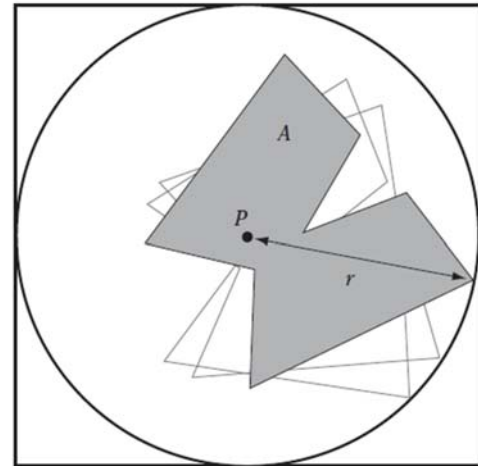


Figure 4.4 (a) AABBs A and B in world space. (b) The AABBs in the local space of A. (c) The AABBs in the local space of B.

- ▶ Nonaligned or (freely) oriented bounding volumes
 - ▶ Some bounding volumes, such as spheres or convex hulls, naturally transform into any coordinate system, as they are not restricted to specific orientations.
- ▶ Aligned bounding volumes
 - ▶ Some bounding volumes such as AABBs are restricted in what orientations they can assume.
 - ▶ The aligned bounding volumes must be realigned as they become unaligned due to object rotation during motion.

AABB from the Object Bounding Sphere

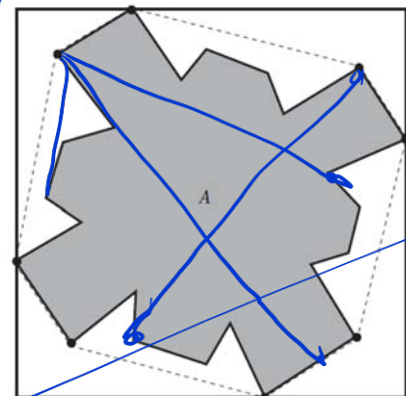
- ▶ Make an AABB large enough to contain the object at any orientation.
- ▶ This fixed-size encompassing AABB is computed as the bounding box of the bounding sphere of the contained object.
- ▶ This method completely circumvents the need to reshape the AABB.
- ▶ Bounding spheres should be considered a potential better choice of bounding volume in this case.



▶ 9

AABB Reconstructed from Original Point Set (1)

- ▶ Except for the previous method, an AABB is dynamically resized as it is being realigned with the coordinate system axes.
- ▶ A straightforward way to construct an AABB is to find six extreme points, two for each coordinate axis.
- ▶ This $O(n)$ algorithm can be expensive if it is performed at runtime for large n .
- ▶ If an object is concave, we can find a convex hull first and store the vertices included in the convex hull in the k first elements for the object.
- ▶ An $O(\log n)$ algorithm also exists, which uses a precomputed search structure.



▶ 10

Sorting $\rightarrow$ search

binary search

$O(N)$
linear search

AABB Reconstructed from Original Point Set (1)

```
// Returns indices imin and imax into pt[] array of the least and
// most, respectively, distant points along the direction dir
void ExtremePointsAlongDirection(Vector dir, Point pt[], int n, int *imin, int *imax)
{
    float minproj = FLT_MAX, maxproj = -FLT_MAX;
    for (int i = 0; i < n; i++) {
        // Project vector from origin to point onto direction vector
        float proj = Dot(pt[i], dir);
        // Keep track of least distant point along direction vector
        if (proj < minproj) {
            minproj = proj;
            *imin = i;
        }
        // Keep track of most distant point along direction vector
        if (proj > maxproj) {
            maxproj = proj;
            *imax = i;
        }
    }
}
```

$O(N)$

► 11

AABB from Hill-climbing Vertices of the Object Representation (1)

- ▶ Another way to speed up the AABB realignment process is to use an object representation in which neighboring vertices of a vertex can be found quickly.
- ▶ Such a representation allows the extreme vertices that define the new AABB to be located through simple hill climbing.

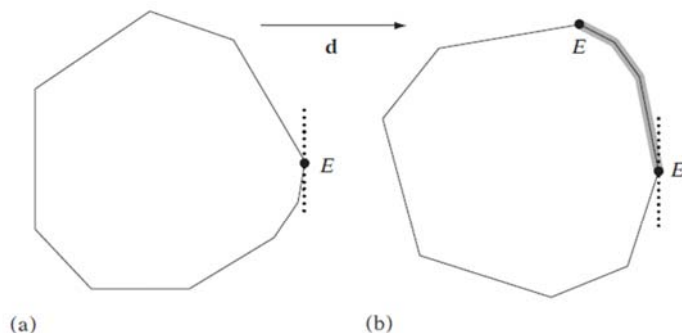


Figure 4.7 (a) The extreme vertex E in direction $\mathbf{d}$. (b) After object rotates counterclockwise, the new extreme vertex E' in direction $\mathbf{d}$ can be obtained by hill climbing along the vertex path highlighted in gray.

► 12

AABB from Hill-climbing Vertices of the Object Representation (2)

- ▶ Instead of keeping track of the minimum and maximum extent values along each axis, six vertex pointers are maintained.
- ▶ These now actually point at the (up to six) extremal vertices of the object along each axis direction.
- ▶ The hill-climbing step now proceeds by comparing the referenced vertices against their neighbor vertices to see if they are still extremal in the same direction as before.
- ▶ So as not to get stuck in local minima, the hill-climbing process requires objects to be convex.
- ▶ For this reason, hill climbing is performed on precalculated convex hulls of nonconvex objects.
- ▶ Overall, this recalculation of the tight AABB is an expected constant-time operation.

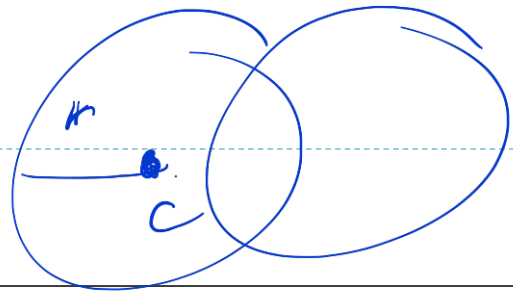
AABB Recomputed from Rotated AABB

- ▶ The most common approach is to simply wrap the rotated AABB itself in a new AABB.
- ▶ This produces an approximate rather than a tight AABB.
- ▶ As the resulting AABB is larger than the one that was started with, the approximate AABB must be computed from a rotation of the original local-space AABB.
- ▶ See the text for function codes.

Bounding Spheres

- ▶ Another very common bounding volume
- ▶ Rivals the axis-aligned bounding box in popularity.
- ▶ Like AABBs, spheres have an inexpensive intersection test.
- ▶ Spheres also have the benefit of being rotationally invariant, which means that they are trivial to transform: they simply have to be translated to their new position.
- ▶ The most memory-efficient bounding volume.
- ▶ Computing an optimal bounding sphere is not as easy as computing an optimal axis-aligned bounding box.

▶ 15



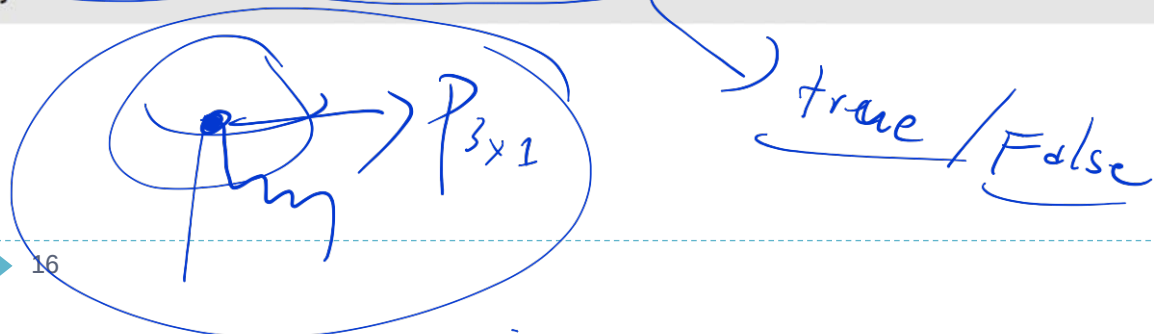
Sphere-Sphere Intersection

test case
AABB
OBB

- ▶ Very simple

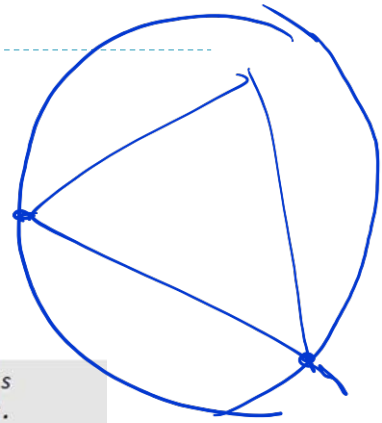
```
int TestSphereSphere(Sphere a, Sphere b)
{
    // Calculate squared distance between centers
    Vector d = a.c - b.c;
    float dist2 = Dot(d, d);
    // Spheres intersect if squared distance is less than squared sum of radii
    float radiusSum = a.r + b.r;
    return dist2 <= radiusSum * radiusSum;
}
```

▶ 16



Computing a Bounding Sphere (1)

- ▶ Two-pass algorithm
 - ▶ Find a good initial almost-bounding sphere
 - ▶ Improve the sphere until it bounds all points.
 - ▶ Results in a tight fit
- ▶ The first pass



*// Compute indices to the two most separated points of the (up to) six points
// defining the AABB encompassing the point set. Return these as min and max.*

```
void MostSeparatedPointsOnAABB(int &min, int &max, Point pt[], int numPts)
{
    // First find most extreme points along principal axes
    int minx = 0, maxx = 0, miny = 0, maxy = 0, minz = 0, maxz = 0;
    for (int i = 1; i < numPts; i++) {
        if (pt[i].x < pt[minx].x) minx = i;
        if (pt[i].x > pt[maxx].x) maxx = i;
        if (pt[i].y < pt[miny].y) miny = i;
        if (pt[i].y > pt[maxy].y) maxy = i;
        if (pt[i].z < pt[minz].z) minz = i;
        if (pt[i].z > pt[maxz].z) maxz = i;
    }
}
```

17

Computing a Bounding Sphere (2)

```
// Compute the squared distances for the three pairs of points
float dist2x = Dot(pt[maxx] - pt[minx], pt[maxx] - pt[minx]);
float dist2y = Dot(pt[maxy] - pt[miny], pt[maxy] - pt[miny]);
float dist2z = Dot(pt[maxz] - pt[minz], pt[maxz] - pt[minz]);
// Pick the pair (min,max) of points most distant
min = minx;
max = maxx;
if (dist2y > dist2x && dist2y > dist2z) {
    max = maxy;
    min = miny;
}
if (dist2z > dist2x && dist2z > dist2y) {
    max = maxz;
    min = minz;
}

void SphereFromDistantPoints(Sphere &s, Point pt[], int numPts)
{
    // Find the most separated point pair defining the encompassing AABB
    int min, max;
    MostSeparatedPointsOnAABB(min, max, pt, numPts);

    // Set up sphere to just encompass these two points
    s.c = (pt[min] + pt[max]) * 0.5f;
    s.r = Dot(pt[max] - s.c, pt[max] - s.c);
    s.r = Sqrt(s.r);
}
```

18

Computing a Bounding Sphere (3)

► The second pass

```
// Given Sphere s and Point p, update s (if needed) to just encompass p
void SphereOfSphereAndPt(Sphere &s, Point &p)
{
    // Compute squared distance between point and sphere center
    Vector d = p - s.c;
    float dist2 = Dot(d, d);
    // Only update s if point p is outside it

    if (dist2 > s.r * s.r) {
        float dist = Sqrt(dist2);
        float newRadius = (s.r + dist) * 0.5f;
        float k = (newRadius - s.r) / dist;
        s.r = newRadius;
        s.c += d * k;
    }
}
```

► 19

deformable object

Computing a Bounding Sphere (4)

► The full code

```
void RitterSphere(Sphere &s, Point pt[], int numPts)
{
    // Get sphere encompassing two approximately most distant points
    SphereFromDistantPoints(s, pt, numPts);

    // Grow sphere to include all points
    for (int i = 0; i < numPts; i++)
        SphereOfSphereAndPt(s, pt[i]);
}
```

► 20

Bounding Sphere from Direction of Maximum Spread (1)

$O(\infty)$

- ▶ Another approach is to analyze the point cloud using statistical methods to find its direction of maximum spread.
- ▶ Given this direction, the two points farthest away from each other when projected onto this axis can be used to determine the center and radius of the starting sphere.

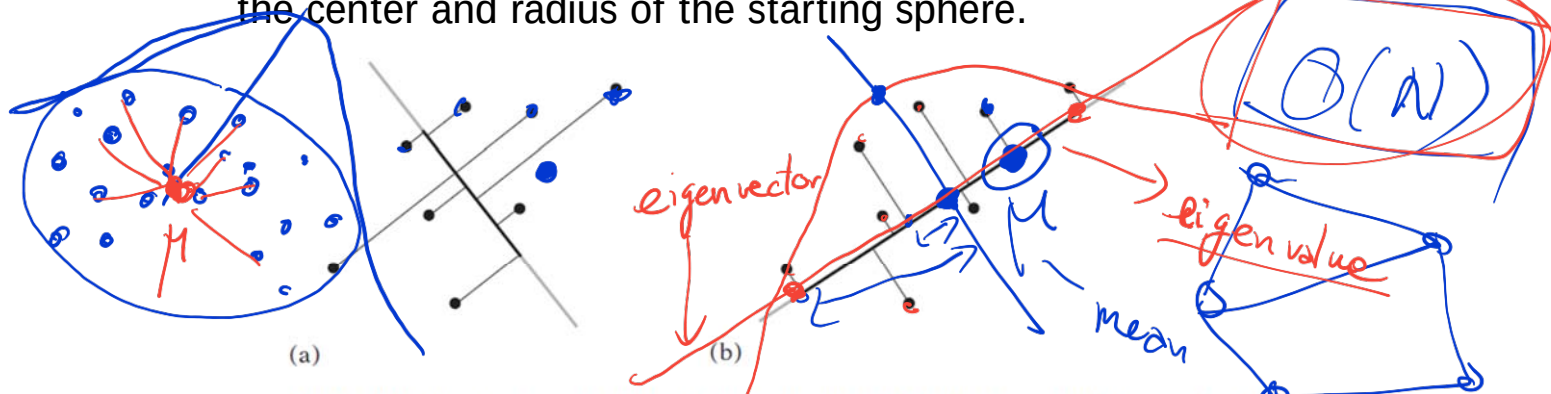


Figure 4.8 The same point cloud projected onto two different axes. In (a) the spread on the axis is small. In (b) the spread is much larger. A bounding sphere can be determined from the axis for which the projected point set has the maximum spread.

▶ 21

PCA (principal component analysis)

Bounding Sphere from Direction of Maximum Spread (2)

- ▶ **Covariance:** Identifies how the variables co-vary

The covariance matrix $C = [c_{ij}]$ for a collection of n points $P_1, P_2, \dots, P_n$ is given by

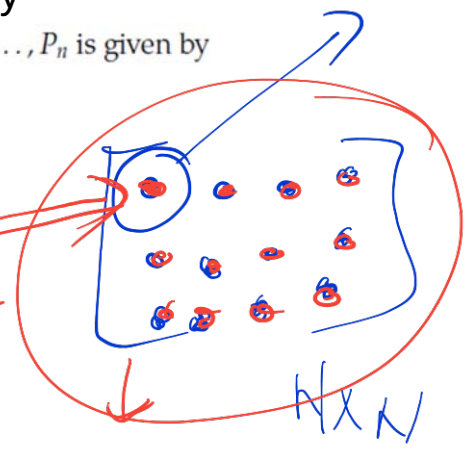
$$c_{ij} = \frac{1}{n} \sum_{k=1}^n (P_{k,i} - u_i)(P_{k,j} - u_j),$$

or equivalently by

$$c_{ij} = \frac{1}{n} \left(\sum_{k=1}^n P_{k,i} P_{k,j} \right) - u_i u_j.$$

The u_i (and u_j) term is the mean of the i -th coordinate value of the points, given by

$$u_i = \frac{1}{n} \sum_{k=1}^n P_{k,i}.$$



$O(\infty)$
 $O(N)$

eigen vector
eigen value
Covariance matrix

OBB

▶ 22

Bounding Sphere from Direction of Maximum Spread (3)

- ▶ The covariance matrix can be decomposed to find the principal directions of the variance.
- ▶ This decomposition is performed by computing the eigenvalues and eigenvectors of the matrix.
- ▶ The eigenvector associated with the largest magnitude eigenvalue corresponds to the axis along which the point data has the largest variance.
- ▶ Similarly, the eigenvector associated with the smallest magnitude eigenvalue is the axis along which the data has the least variance.
- ▶ Look at the text for codes to compute a covariance matrix, eigenvalues, and eigenvectors.

▶ 23

$O(N^2)$
 $O(N)$

PCA
4 sub
space
linear
algebra

Bounding Sphere from Direction of Maximum Spread (4)

- ▶ Computing a sphere from the two most distant points

```
void EigenSphere(Sphere &eigSphere, Point pt[], int numPts)
{
    Matrix33 m, v;

    // Compute the covariance matrix m
    CovarianceMatrix(m, pt, numPts);
    // Decompose it into eigenvectors (in v) and eigenvalues (in m)
    Jacobi(m, v);

    // Find the component with largest magnitude eigenvalue (largest spread)
    Vector e;
    int maxc = 0;
    float maxf, maxe = Abs(m[0][0]);
    if ((maxf = Abs(m[1][1])) > maxe) maxc = 1, maxe = maxf;
    if ((maxf = Abs(m[2][2])) > maxe) maxc = 2, maxe = maxf;
    e[0] = v[0][maxc];
    e[1] = v[1][maxc];
    e[2] = v[2][maxc];

    // Find the most extreme points along direction 'e'
```

▶ 24

→ code

Bounding Sphere from Direction of Maximum Spread (5)

```
int imin, imax;
ExtremePointsAlongDirection(e, pt, numPts, &imin, &imax);
Point minpt = pt[imin];
Point maxpt = pt[imax];

float dist = Sqrt(Dot(maxpt - minpt, maxpt - minpt));
eigSphere.r = dist * 0.5f;
eigSphere.c = (minpt + maxpt) * 0.5f;
}
```

- ▶ The final code is

```
void RitterEigenSphere(Sphere &s, Point pt[], int numPts)
{
    // Start with sphere from maximum spread
    EigenSphere(s, pt, numPts);

    // Grow sphere to include all points
    for (int i = 0; i < numPts; i++)
        SphereOfSphereAndPt(s, pt[i]);
}
```

▶ 25

Bounding Sphere Through Iterative Refinement (1)

- ▶ The previous algorithms begins with an initial sphere and refine the sphere iteratively.
- ▶ Given a better initial sphere, the final sphere can be expected to be better (better fit) as well.
- ▶ Consequently, we can feed the output of the algorithm to an input to the algorithm in an iterative manner.
- ▶ The resulting sphere of one iteration is simply shrunk by a small amount to make it an underestimate for the next iterative call.
- ▶ To further improve the results, the points are considered at random, rather than in the same order from iteration to iteration.
- ▶ The resulting sphere is usually much better than that produced by the previous method using the maximum spread at the cost of a few extra iterations over the input data.
- ▶ If the same iterative approach is applied to the previous method using the maximum spread, the results are comparable.

▶ 26

Bounding Sphere Through Iterative Refinement (2)

```
void RitterIterative(Sphere &s, Point pt[], int numPts)
{
    const int NUM_ITER = 8;

    RitterSphere(s, pt, numPts);
    Sphere s2 = s;
    for (int k = 0; k < NUM_ITER; k++) {
        // Shrink sphere somewhat to make it an underestimate (not bound)
        s2.r = s2.r * 0.95f;

        // Make sphere bound data again
        for (int i = 0; i < numPts; i++) {
            // Swap pt[i] with pt[j], where j randomly from interval [i+1,numPts-1]
            DoRandomSwap();
            SphereOfSphereAndPt(s2, pt[i]);
        }

        // Update s whenever a tighter sphere is found
        if (s2.r < s.r) s = s2;
    }
}
```

► 27

The Minimum Bounding Sphere (1)

- Brute-force algorithm
 - A sphere is uniquely defined by four (non co-planar) points.
 - For a set of points, we can consider all possible combinations of four (then three, then two) points, compute the smallest sphere through these points, and keep the sphere if it contains all other points.
 - The kept sphere with the smallest radius is then the minimum bounding sphere.
 - Complexity: $O(n^5)$
- There exists a randomized algorithm that runs in expected linear time for minimum bounding sphere computation [Welzl91]..
 - Assume a minimum bounding sphere S has been computed for a point set P .
 - If a new point Q is added to P , then only if Q lies outside S , S needs to be recomputed.
 - Q must lie on the boundary of the new minimum bounding sphere for the point set $P \cup \{Q\}$.

► 28

The Minimum Bounding Sphere (2)

```
Sphere WelzlSphere(Point pt[], unsigned int numPts, Point sos[], unsigned int numSos)
{
    // if no input points, the recursion has bottomed out. Now compute an
    // exact sphere based on points in set of support (zero through four points)
    if (numPts == 0) {
        switch (numSos) {
            case 0: return Sphere();
            case 1: return Sphere(sos[0]);
            case 2: return Sphere(sos[0], sos[1]);
            case 3: return Sphere(sos[0], sos[1], sos[2]);
            case 4: return Sphere(sos[0], sos[1], sos[2], sos[3]);
        }
    }
    // Pick a point at "random" (here just the last point of the input set)
    int index = numPts - 1;
    // Recursively compute the smallest bounding sphere of the remaining points
    Sphere smallestSphere = WelzlSphere(pt, numPts - 1, sos, numSos); // (*)
    // If the selected point lies inside this sphere, it is indeed the smallest
    if(PointInsideSphere(pt[index], smallestSphere))
        return smallestSphere;
    // Otherwise, update set of support to additionally contain the new point
    sos[numSos] = pt[index];
    // Recursively compute the smallest sphere of remaining points with new s.o.s.
    return WelzlSphere(pt, numPts - 1, sos, numSos + 1);
}
```

► 29

Homework

- Take an example that illustrates how WelzlSphere() works. You need to explain how the recursive behaviors of WelzlSphere() step by step.
- Due date: One week later

► 30